

P2909R0

Dude, where's my char?

Published Proposal, 2023-07-29

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

1 Introduction

2 Proposal

3 Wording

4 Impact on existing code

5 Implementation

References

Informative References

"In character, in manner, in style, in all things, the supreme excellence is simplicity." — Henry Wadsworth Longfellow

§ 1. Introduction

The C++20 formatting facility (`std::format`) allows formatting of `char` as an integer via format specifiers such as `d` and `x`. Unfortunately [P0645] that introduced the facility didn't take into account that signedness of `char` is implementation-defined and specified this formatting in terms of `to_chars` with the value implicitly converted (promoted) to `int`. This had some undesirable effects discovered after getting usage experience and resolved in the `{fmt}` library ([FMT]). This paper proposes applying a similar fix to `std::format`.

First, `std::format` normally produces consistent output across platforms for the same integral types and the same IEEE 754 floating point types. Formatting `char` as an integer breaks this nice property making the output implementation-defined even if the `char` size is effectively the same.

Second, `char` is used as a code unit type in `std::format` and other text processing facilities. In these use cases one normally needs to either output `char` as (a part of) text which is the default or as a bit pattern. Having it sometimes be output as a signed integer is surprising to users. It is particularly surprising when formatted in a non-decimal base. For example, assuming UTF-8 literal encoding:

```
for (char c : std::string("👤")) {
    std::print("\\x{:02x}", c);
}
```

will print either

```
\xf0\x9f\xa4\xb7
```

or

```
\x-10\x-61\x-5c\x-49
```

depending on a platform. Since it is implementation-defined, the user may not even be aware of this issue which can then manifest itself when the code is compiled and run on a different platform or with different compiler flags.

This particular case can be fixed by adding a cast to `unsigned char` but it may not be as easy to do when formatting ranges compared to using format specifiers.

§ 2. Proposal

This paper proposes making code unit types formatted as unsigned integers instead of implementation-defined.

Code	Before	After
<pre>// Assuming UTF-8 as a literal encoding. for (char c : std::string("👤")) { std::print("\\x{:02x}", c); }</pre>	<pre>\xf0\x9f\xa4\xb7</pre>	<pre>\xf0\x9f\xa4\xb7</pre>
	or	
	<pre>\x-10\x-61\x-5c\x-49</pre> (implementation-defined)	

§ 3. Wording

Change in `[tab:format.type.char]`:

Table 69: Meaning of type options for `charT` `[tab:format.type.char]`

Type	Meaning
none, <code>c</code>	Copies the character to the output.
<code>b</code> , <code>B</code> , <code>d</code> , <code>o</code> , <code>x</code> , <code>X</code>	As specified in Table 68 with <code>value</code> converted to the corresponding unsigned type .
<code>?</code>	Copies the escaped character (<code>[format.string.escaped]</code>) to the output.

§ 4. Impact on existing code

This is a breaking change but the it only affects the output of negative/large code units when output via opt-in format specifiers. There were no issues reported when the change was shipped in `{fmt}` and the number of uses of `std::format` is orders of magnitude smaller at the moment.

§ 5. Implementation

The proposed change has been implemented in the `{fmt}` library ([\[FMT\]](#)).

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. *The `fmt` library*. URL: <https://github.com/fmtlib/fmt>

[P0645]

Victor Zverovich. *Text Formatting*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0645r10.html>